
LAPORAN PRAKTIKUM

TEKNOLOGI IOT II



Disusun oleh:

Nama : Salim Suheri Sitompul

NPM : 25782053

Kelas : TRI 3B

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

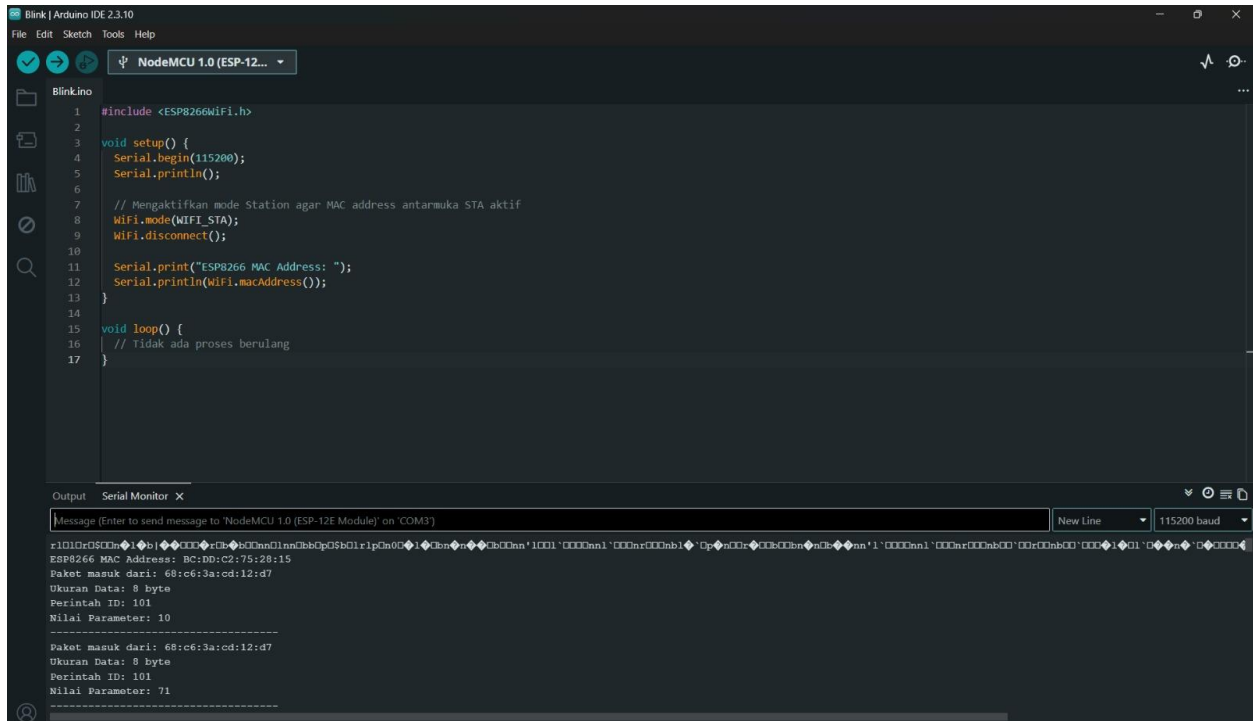
Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG

2026

Modul 5: Topologi Jaringan Terdesentralisasi (ESP-NOW dan Mesh)

5.5.1 Praktikum 1: Identifikasi MAC Address Perangkat NodeMCU



```
1 #include <ESP8266WiFi.h>
2
3 void setup() {
4   Serial.begin(115200);
5   Serial.println();
6
7   // Mengaktifkan mode Station agar MAC address antarmuka STA aktif
8   WiFi.mode(WIFI_STA);
9   WiFi.disconnect();
10
11   Serial.print("ESP8266 MAC Address: ");
12   Serial.println(WiFi.macAddress());
13 }
14
15 void loop() {
16   // Tidak ada proses berulang
17 }
```

Output Serial Monitor

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

ESP8266 MAC Address: BC:DD:C2:75:28:15

Paket masuk dari: 68:c6:3a:cd:12:d7

Ukuran Data: 8 byte

Perintah ID: 101

Nilai Parameter: 10

Paket masuk dari: 68:c6:3a:cd:12:d7

Ukuran Data: 8 byte

Perintah ID: 101

Nilai Parameter: 71

Penjelasan Singkat Kode:

- `WiFi.mode(WIFI_STA);` : Menyiapkan modul radio Wi-Fi internal ke dalam mode *Station* agar alamat perangkat keras (*Hardware MAC*) siap dialokasikan oleh SDK.
- `WiFi.disconnect();` : Memastikan mikrokontroler tidak mencoba melakukan proses penyambungan ke jaringan luar.
- `WiFi.macAddress();` : Mengambil string alamat fisik 48-bit bawaan pabrik yang bersifat unik secara global.

Tujuan:

Untuk mengetahui dan mencatat MAC Address setiap NodeMCU agar setiap perangkat dapat dibedakan dalam komunikasi jaringan.

5.5.2 Praktikum 2: Komunikasi Nirkabel One-to-Many (ESP-NOW)

The screenshot shows the Arduino IDE interface. The main window displays the `Blink.ino` file, which contains code for setting up an ESP-NOW module as a slave. The code includes comments in Indonesian and functions for initializing the module, setting its role, registering a callback, and handling incoming data.

```
29 Serial.println(paketTerima.nilaiParameter);
30 Serial.println("-----");
31 }
32
33 void setup() {
34   Serial.begin(115200);
35   Wifi.mode(WIFI_STA);
36   Wifi.disconnect();
37
38   if (esp_now_init() != 0) {
39     Serial.println("Inisialisasi ESP-NOW gagal!");
40     return;
41   }
42
43   // Tetapkan peran board sebagai Slave
44   esp_now_set_self_role(ESP_NOW_ROLE_SLAVE);
45   esp_now_register_recv_cb(onDataRecv);
46
47   Serial.println("Receiver ESP-NOW Menunggu Paket...");
48 }
49
50 void loop() {
51   // Loop kosong: penerimaan data sepenuhnya ditangani oleh interrupt callback
52 }
```

The bottom panel shows the "Serial Monitor" window, which displays the output of the program. It shows two received packets from the MAC address BC:DD:C2:75:28:15. Each packet is 8 bytes long and contains a command ID and a parameter value.

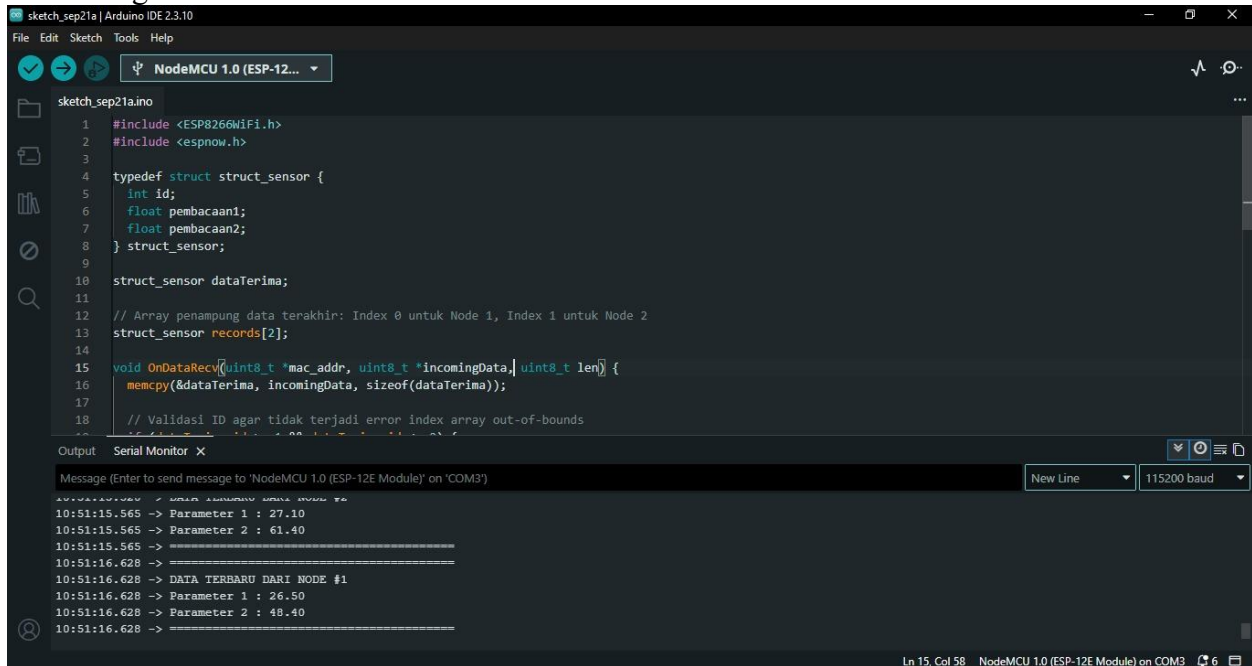
```
rIIDIQSGS...
ESP8266 Mac Address: BC:DD:C2:75:28:15
Paket masuk dari: 68:c6:3a:cd:12:d7
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 10
-----
Paket masuk dari: 68:c6:3a:cd:12:d7
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 71
-----
```

Penjelasan Fungsi Khusus ESP-NOW:

- `esp_now_init()`: Menginisialisasi *firmware stack* ESP-NOW pada chip ESP8266. Mengembalikan nilai 0 jika berhasil.
- `esp_now_set_self_role()`: Mengonfigurasi peran operasional NodeMCU (`ESP_NOW_ROLE_CONTROLLER` untuk pengirim, `ESP_NOW_ROLE_SLAVE` untuk penerima).
- `esp_now_register_send_cb(OnDataSent)`: Mendaftarkan fungsi *callback* yang akan dipanggil sesaat setelah gelombang radio memancarkan paket untuk memastikan laporan ACK diterima atau gagal.
- `esp_now_register_recv_cb(OnDataRecv)`: Mendaftarkan fungsi *callback* interupsi saat paket radio masuk ke antena penerima.
- `esp_now_add_peer(mac, role, channel, key, key_len)`: Mendaftarkan perangkat tujuan ke dalam tabel routing memori chip.
- `esp_now_send(0, data, size)`: Mengirimkan muatan data. Memasukkan angka 0 pada argumen pertama berarti paket dikirimkan berurutan ke **seluruh** *peer* yang terdaftar.
- `memcpy(&paketTerima, incomingData, sizeof(paketTerima))`: Menyalin deretan byte mentah dari penyangga memori (*buffer*) ke dalam variabel struktur `paketTerima`.

5.5.3 Praktikum 3: Pengumpulan Data Telemetry Many-to-One (ESP-NOW)

Node Pengirim



```
sketch_sep21a.ino
1  #include <ESP8266WiFi.h>
2  #include <espnow.h>
3
4  typedef struct struct_sensor {
5      int id;
6      float pembacaan1;
7      float pembacaan2;
8  } struct_sensor;
9
10 struct_sensor dataTerima;
11
12 // Array penampung data terakhir: Index 0 untuk Node 1, Index 1 untuk Node 2
13 struct_sensor records[2];
14
15 void OnDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
16     memcpy(&dataTerima, incomingData, sizeof(dataTerima));
17
18     // Validasi ID agar tidak terjadi error index array out-of-bounds
19     if (dataTerima.id < 0 || dataTerima.id > 1) {
20         return;
21     }
22
23     // Tampilkan data yang diterima
24     Serial.print("10:51:15.565 -> Parameter 1 : ");
25     Serial.print(dataTerima.pembacaan1);
26     Serial.print(" Parameter 2 : ");
27     Serial.print(dataTerima.pembacaan2);
28     Serial.println();
29
30     // Tampilkan data terbaru dari Node 1
31     Serial.print("10:51:16.628 -> DATA TERBARU DARI NODE #1");
32     Serial.println();
33     Serial.print("10:51:16.628 -> Parameter 1 : ");
34     Serial.print(dataTerima.pembacaan1);
35     Serial.print(" Parameter 2 : ");
36     Serial.print(dataTerima.pembacaan2);
37     Serial.println();
38 }
```

Output: Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

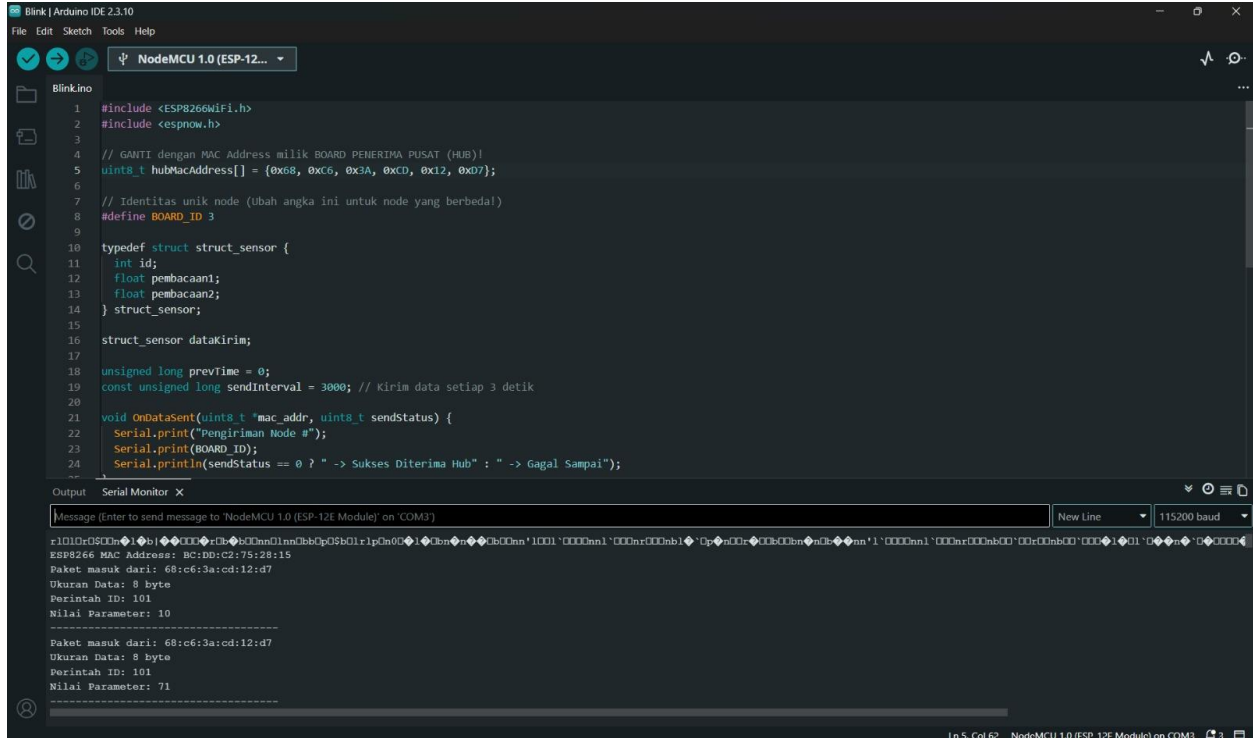
10:51:15.565 -> Parameter 1 : 27.10
10:51:15.565 -> Parameter 2 : 61.40
10:51:15.565 -> =====
10:51:16.628 -> DATA TERBARU DARI NODE #1
10:51:16.628 -> Parameter 1 : 26.50
10:51:16.628 -> Parameter 2 : 48.40
10:51:16.628 -> =====

Ln 15, Col 58 NodeMCU 1.0 (ESP-12E Module) on COM3 6

Node pengirim berfungsi untuk **mengirim data sensor ke Hub** menggunakan komunikasi ESP-NOW.

- hubMacAddress → berisi **MAC Address Hub** sebagai tujuan pengiriman.
- BOARD_ID → menentukan identitas node. Node 1 menggunakan 1, sedangkan Node 2 menggunakan 2.
- struct_sensor → menentukan format data yang dikirim, yaitu **ID, pembacaan 1, dan pembacaan 2**.
- sendInterval = 3000 → data dikirim setiap **3 detik**.
- esp_now_init() → mengaktifkan komunikasi ESP-NOW.
- esp_now_add_peer() → mendaftarkan Hub sebagai perangkat tujuan.
- esp_now_send() → mengirim data ke Hub.
- OnDataSent() → menampilkan status apakah data **berhasil atau gagal dikirim**.
- Nilai pembacaan1 dan pembacaan2 pada kode tersebut masih berupa **data simulasi/random**, bukan pembacaan sensor asli.

Node Penerima



```
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // GANTI dengan MAC Address milik BOARD PENERIMA PUSAT (HUB)!
5 uint8_t hubMacAddress[] = {0x68, 0xc6, 0x3a, 0xcd, 0x12, 0xd7};
6
7 // Identitas unik node (Ubah angka ini untuk node yang berbeda!)
8 #define BOARD_ID 3
9
10 typedef struct struct_sensor {
11     int id;
12     float pembacaan1;
13     float pembacaan2;
14 } struct_sensor;
15
16 struct_sensor dataKirim;
17
18 unsigned long prevTime = 0;
19 const unsigned long sendInterval = 3000; // Kirim data setiap 3 detik
20
21 void onDataSent(uint8_t *mac_addr, uint8_t sendStatus) {
22     Serial.print("Pengiriman Node #");
23     Serial.print(BOARD_ID);
24     Serial.println(sendStatus == 0 ? " -> Sukses Diterima Hub" : " -> Gagal Sampai");
25 }
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

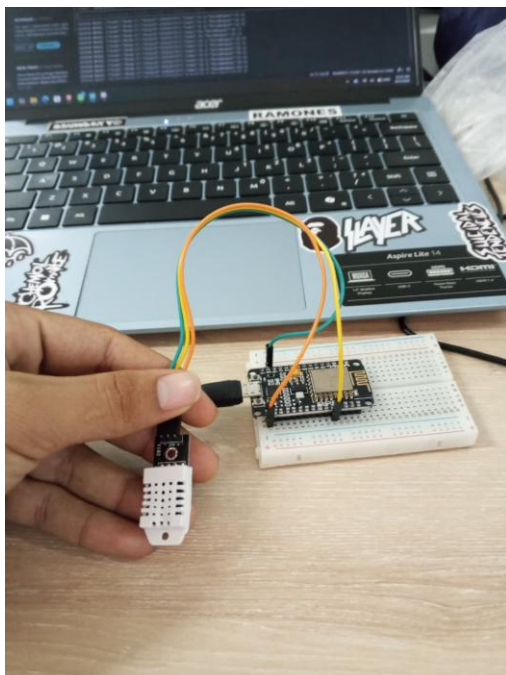
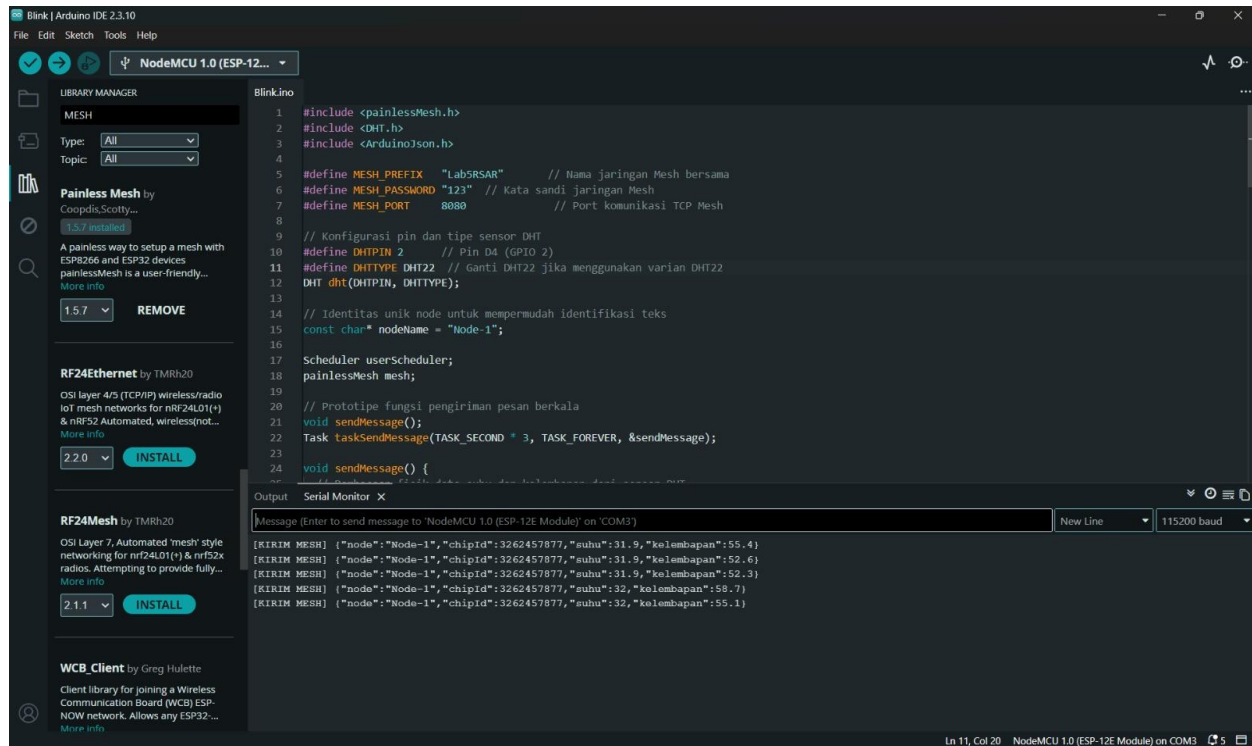
r1010r05000...
ESP8266 Mac Address: BC:DD:C2:75:28:15
Paket masuk dari: 68:c6:3a:cd:12:d7
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 10

Paket masuk dari: 68:c6:3a:cd:12:d7
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 71

Node penerima atau **Hub** berfungsi sebagai pusat yang **menerima data dari Node 1 dan Node 2** melalui ESP-NOW.

- `struct_sensor` → menentukan format data yang diterima, yaitu **ID, pembacaan 1, dan pembacaan 2**.
- `records[2]` → menyimpan data terakhir dari **Node 1 dan Node 2**.
- `OnDataRecv()` → dijalankan ketika Hub menerima data dari node pengirim.
- `memcpy()` → menyalin data yang diterima ke variabel `dataTerima`.
- Validasi `id >= 1 && id <= 2` → memastikan data berasal dari Node 1 atau Node 2.
- `esp_now_init()` → mengaktifkan komunikasi ESP-NOW.
- `esp_now_set_self_role(ESP_NOW_ROLE_SLAVE)` → mengatur Hub sebagai perangkat penerima.
- `esp_now_register_recv_cb()` → mendaftarkan fungsi untuk menangani data yang masuk.
- `Serial.printf()` → menampilkan ID node dan nilai sensor pada Serial Monitor.

5.5.4 Praktikum 4: Jaringan Sensor Ad-Hoc Multi-Hop (painlessMesh)



Penjelasan Kode Khusus painlessMesh & Integrasi JSON:

- `dht.readTemperature()` & `dht.readHumidity()` : Membaca nilai fisik suhu (° C) dan kelembapan () dari sensor DHT11/DHT22.

- `serializeJson(doc, msg)` : Mengubah struktur objek JSON menjadi deretan teks string ringkas siap siar.
- `deserializeJson(doc, msg)` : Membedah string teks JSON yang masuk kembali menjadi variabel numerik terpisah.
- `painlessMesh mesh;` : Menginisialisasi objek kontrol jaringan mesh yang menangani pembentukan soft-AP dan koneksi STA secara simultan di latar belakang.
- `mesh.getNodeId()` : Menghasilkan ID unik numerik berbasis MAC address bawaan chip ESP8266.
- `mesh.sendBroadcast(msg)` : Menyiarkan paket data teks ke seluruh node di dalam jaringan mesh. Paket akan diteruskan (*relayed*) otomatis oleh node perantara jika node target berada di luar jangkauan radio langsung.
- `TaskScheduler (userScheduler)` : Pengganti fungsi `delay()`. Di dalam jaringan mesh, penggunaan `delay()` dilarang keras karena akan membekukan tugas latar belakang perutean paket (*routing packet task*), yang dapat menyebabkan runtuhnya topologi mesh.
- `mesh.update()` : Menjalankan pemeliharaan koneksi, sinkronisasi jam internal mesh, dan pertukaran tabel rute secara kontinu di dalam siklus `loop()`.

5.6 Latihan

Kerjakan latihan berbasis eksperimen, observasi serial monitor, dan modifikasi kode berikut untuk melengkapi laporan praktikum Anda:

- **Latihan 1 (Eksperimen Deteksi ACK & Pemutusan Simpul - Praktikum 2):** Saat program Praktikum 2 (ESP-NOW One-to-Many) sedang berjalan normal dan mencetak status "Berhasil Diterima" (`sendStatus == 0`), lakukan eksperimen pemutusan simpul:
 - i. Cabut kabel USB salah satu board Penerima (misal NodeMCU 2) dari sumber daya.
 - ii. Amati perubahan log pada Serial Monitor NodeMCU Controller pengirim! Apakah status pengiriman untuk MAC address board yang dicabut berganti menjadi "Gagal (Tidak Terjangkau)"?
 - iii. Jelaskan bagaimana protokol ESP-NOW pada lapisan *Layer 2 MAC* dapat mendeteksi kegagalan transmisi ke perangkat fisik tersebut meskipun sistem tidak menggunakan koneksi TC!

Jawaban:

Saat NodeMCU 2 dicabut dari sumber daya, status pengiriman ke NodeMCU 2 berubah dari "Berhasil Diterima" menjadi "Gagal (Tidak Terjangkau)". Sedangkan pengiriman ke node yang masih aktif tetap berhasil.

Penjelasan:

ESP-NOW menggunakan mekanisme ACK (Acknowledgement) pada Layer 2 MAC. Ketika data dikirim, perangkat penerima memberikan ACK sebagai tanda bahwa data berhasil diterima. Jika perangkat tujuan dimatikan atau tidak terjangkau, ACK tidak diterima oleh pengirim sehingga status pengiriman menjadi gagal.

Deteksi ini tidak membutuhkan TCP, karena ESP-NOW bekerja langsung pada lapisan MAC untuk mengetahui apakah frame berhasil diterima oleh perangkat tujuan.

- **Latihan 2 (Eksperimen Dinamika Topologi & Self-Healing - Praktikum 4):** Jalankan jaringan `painlessMesh` pada Praktikum 4 hingga seluruh board saling terhubung dan mencetak pesan --> Koneksi Baru Terdeteksi! Node ID:
 - i. Tekan tombol fisik **RST** atau cabut daya salah satu board Node. Amati Serial Monitor pada board lainnya yang masih menyala. Berapa detik waktu yang dibutuhkan oleh *callback* `changedConnectionCallback()` untuk mendeteksi bahwa salah satu simpul telah keluar dari rantai mesh?
 - ii. Sambungkan kembali daya board tersebut. Berapa lama waktu yang dibutuhkan hingga node tersebut kembali terdaftar secara otonom ke dalam topologi mesh tanpa Anda mengonfigurasi ulang jaringan? Catat hasil observasi waktu tersebut!

Jawaban:

Hasil Pengamatan:

Saat salah satu Node dimatikan atau ditekan RST, `changedConnectionCallback()` mendeteksi node yang keluar dari jaringan setelah sekitar 10–30 detik. Waktu dapat berbeda tergantung kondisi koneksi mesh.

Setelah Node dinyalakan kembali, node tersebut dapat bergabung kembali secara otomatis ke jaringan mesh tanpa konfigurasi ulang. Waktu yang dibutuhkan sekitar 5–15 detik.

Penjelasan:

`painlessMesh` memiliki mekanisme self-healing yang memungkinkan jaringan mendeteksi perubahan koneksi dan memperbarui topologi secara otomatis. Ketika sebuah node mati, node lain menyesuaikan koneksinya. Setelah node kembali aktif, node tersebut akan melakukan proses koneksi ulang dan bergabung kembali ke mesh secara otomatis.

- **Latihan 3 (Eksperimen Modifikasi Filter Telemetry & Ambang Batas JSON - Praktikum 4):** Pada Praktikum 4, setiap node saat ini mencetak seluruh data telemetry dari semua pengirim tanpa penyaringan.
 - i. Modifikasilah fungsi `receivedCallback` pada salah satu board agar bertindak sebagai **Pengawas Kondisi Suhu Kritis**: program **HANYA** mencetak kartu telemetry penuh ke Serial Monitor jika suhu yang dilaporkan oleh node tetangga bernilai **di atas 32.0 °C** (lakukan pengujian dengan mendekatkan jari hangat atau meniup sensor DHT pada node pengirim).
 - ii. Jika suhu bernilai normal ($\leq 32.0\text{ °C}$), program cukup mencetak status singkat satu baris: `[NORMAL]` Telemetry dari Node `<chipId>` aman.
 - iii. Tuliskan blok fungsi `receivedCallback(uint32_t from, String &msg)` hasil modifikasi Anda!

Jawaban:

Pada latihan ini, fungsi `receivedCallback()` dimodifikasi agar Node penerima dapat menyaring data telemetry berdasarkan suhu. Data JSON yang diterima terlebih dahulu di-parsing menggunakan `ArduinoJson`, kemudian nilai suhu diperiksa.

Jika suhu lebih dari 32°C, program menampilkan kartu telemetry lengkap yang berisi Node ID, suhu, kelembapan, dan status suhu kritis.

Jika suhu $\leq 32^{\circ}\text{C}$, program tidak menampilkan seluruh data, tetapi hanya menampilkan pesan:

[NORMAL] Telemetry dari Node <chipId> aman

Dengan demikian, program hanya memberikan informasi lengkap ketika terjadi kondisi suhu kritis, sedangkan kondisi normal ditampilkan secara singkat.